Internet of Things and Wireless Networks

Henri Heyden
stu240825

Parham Razaghian
stu258323

Summer Term 2026

To see the sources of this project, visit `https://github.com/CrazyDiamond-75/Internet-of-Things-and-Wireless-Networks/tree/project_submission/Project`. *Some additional programs to help with configuration and compilation are available under the main branch.*

1 Project

In our project we looked at the following tasks.

Task 1: 5 Points Setup one device which relays RSSIs of foreign advertisements to a general purpose computer to convert to distance.

Task 2: 10 Points Setup three devices which relay the RSSIs of one device to a general purpose computer and triangulate the position.

Task 3: 15 Points Calculate a dynamic visualization on the general purpose device to track the movement of the user device.

1.1 Solution for Task 1

For the first task we programmed firmware for one ESP32 chip, which continuously scans for BLE traffic. If a message is received, the following payload is redirected via a central-peripheral connection to my laptop.

```
typedef struct __packed
{
    uint8_t nodeID;      // Our node ID. Either 1, 2, or 3.
    int64_t timestamp;   // Used by the peripheral to calculate accurate positioning.
```

```
    int8_t rssi;         // RSSI of the traffic we received.
    bt_addr_le_t sender; // Address of the device which sent the traffic.
} message_t;
```

The laptop then logs the content of all messages received to a file. We then use another python script to evaluate the data and convert it to a distance. The distance calculation is based on the following method.

$$d = 10^{\frac{R_0 - R}{10N}}$$

Here, R_0 is the RSSI at distance one meter, R is the RSSI, and $N \in [2, 4]$ is an environmental parameter. This meant, that to measure the distance to a certain device, we needed to calibrate the equipment, such that the R_0 parameter is known.

This is of course very tedious, as any individual sender-receiver pair would mean a different R_0, which is why we used my headphones and one specific ESP32 to calibrate the system and record data. See Figure 1 for an example plot.

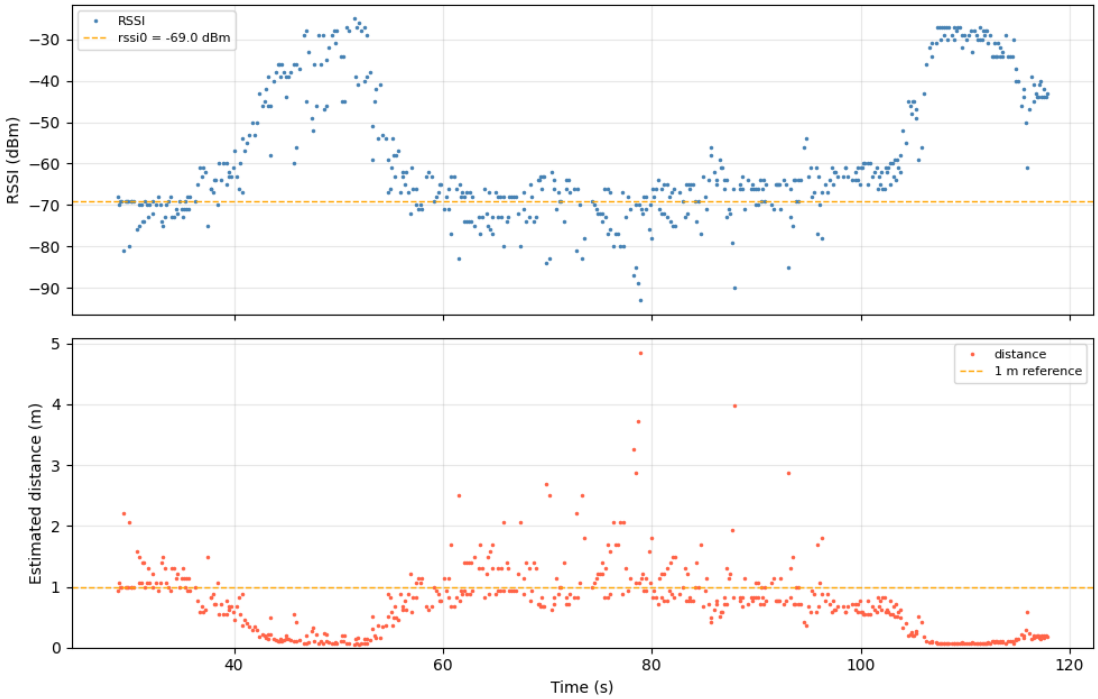

Figure 1: Distance and RSSI between a node and my Bluetooth headphones in pairing mode calculated using the first method.

As seen in Figure 1, once we go beyond one meter, the RSSI becomes pretty noisy. In reality I went from one meter to zero meters and then increased my distance by one meter every time and waited a bit, and finally returned to zero meters at the end. But distances

between zero and one meter are pretty exact. Thus, a system like this, ideally with some passing mean or other denoising filter, can be used for proximity detection.

1.2 Solution for Task 2

For the second task we deployed three nodes in an approximately equidistant triangle with the same firmware, but with different node ids. We then stood before a decision on how to triangulate the position of a device, based on RSSIs of three devices. There were three possible approaches.

1.2.1 Direct Triangulation

We could use the method for task 1 to converts RSSIs to distances, and triangulate using Barycentric coordinates. This would mean calibrating each device/node pair for the devices we want to track, and the three nodes, as it would need RSSIs at distance one meter. All this overhead prompted us to look a bit further.

1.2.2 Distance Ratio Equation

We can rewrite the distance equation to the following

$$R_i = R_{0_i} - 10N \cdot \log_{10}(d_i)$$

for node i. If we assume, that R_0 is the same for all three ESP32s, we can subtract two RSSIs and get

$$\begin{aligned} R_i - R_j &= R_0 - R_0 - 10N \cdot \log_{10}(d_i) + 10N \cdot \log_{10}(d_j) \\ &= 10N \cdot \log_{10}(d_j/d_i) \end{aligned}$$

Thus we obtain

$$d_j/d_i = 10^{\frac{R_i - R_j}{10N}}$$

We name $r_{ji} := d_j/d_i$ and calculate r_{21} and r_{31}. For node positions $x_1, y_1, x_2, y_2, x_3, y_3$ we know

$$d_i = \sqrt{(x - x_i)^2 + (y - y_i)^2}$$

Because $d_2 = r_{21} \cdot d_1$ and $d_3 = r_{31} \cdot d_1$ we have

$$\begin{aligned} \sqrt{(x - x_2)^2 + (y - y_2)^2} &= r_{21} \cdot \sqrt{(x - x_1)^2 + (y - y_1)^2} \\ \sqrt{(x - x_3)^2 + (y - y_3)^2} &= r_{31} \cdot \sqrt{(x - x_1)^2 + (y - y_1)^2} \end{aligned}$$

Squaring both sides yields

$$(x - x_2)^2 + (y - y_2)^2 = r_{21}^2 \cdot ((x - x_1)^2 + (y - y_1)^2)$$
$$(x - x_3)^2 + (y - y_3)^2 = r_{31}^2 \cdot ((x - x_1)^2 + (y - y_1)^2)$$

This a non-linear equation system with two equations and two unknowns x, y. So, we don't need to measure the RSSIs at a fixed distance to configure the system, and we can track multiple devices at the same time. All that is left to do is solve the system.

While this solution is mathematically pretty, it is hard to compute very fast on my laptop. And because we have to first triangulate before applying a smoothing algorithm, this means, that a lot of equation systems have to be solved very fast. This is inoptimal.

1.2.3 Quantized Distance Ratio Approximation

Instead of solving equations using an algorithm to do so, we can also approximate a solution which converts measured distance ratios to device positions. We can easily calculate expected r_{21} and r_{31} from a device position x, y, when the node positions do not change. The idea is to calculate a lookup table which maps the ratios to positions, by calculating the ratios for all positions in a fine grid.

In reality, because we can not have our lookup function go through infinite entries, we have to choose from a finite amount. For this we used a k-d tree, which stores (x, y) pairs in (r_{21}, r_{31}) space. To query the tree, we find a fixed amount of nearest neighbours of the calculated ratios, and compute the interpolated position.

Approximating positions using this method yields good results, as the area inside the triangle of nodes has individual ratio pairs, see Figure 2.

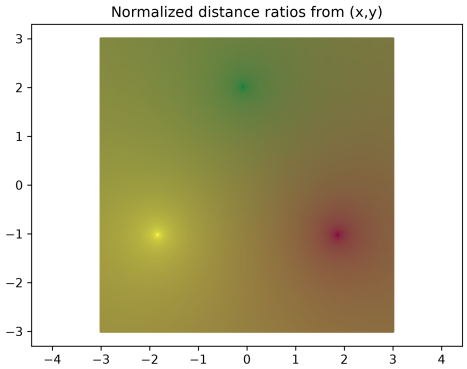

Figure 2: Ratios in colour from positions in an example configuration. Ratios have been normalised for better plotting by applying the natural logarithm.

The algorithm might have some problems when both ratios are near 0.5 in transposed logarithmic space[1], as seen in Figure 3, but realistically this just means, that the device is near the first node.

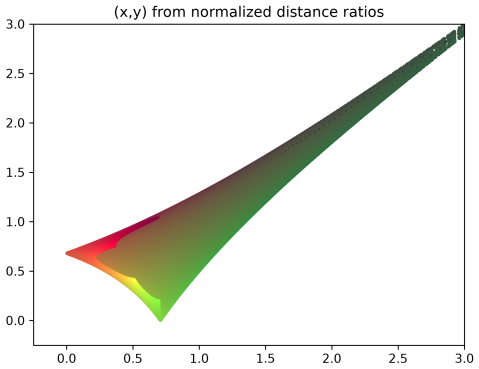

Figure 3: Positions from normalised ratio space in an example configuration. Ratios have been normalised for better plotting by applying the natural logarithm.

Additionally, the solution works actually pretty good in reality.

1.3 Solution for Task 3

We implemented the visualization as another application which reads the triangulated positions from a file and displays them. The triangle which the nodes form is also overlaid. The application can be configured to highlight addresses, which are known to be from a user device. This could be determined through, e.g., the content of the devices' Bluetooth advertisements.

[1] $\log(1 + r_{ji})$.